

Code: inputs and recommendations

Figure 1. Browser screenshot of unchanged source excerpts, including the data model, numeric input validation and category search. Original line numbers are retained.

Video | original source lines 31-39

```

31 @dataclass
32 class Video:
33     title: str
34     category: str    # e.g. Tech, Music, Fitness, Education, Comedy, Gaming, Vlog
35     mood: str        # e.g. Relaxing, Exciting, Informative, Funny
36     duration: int    # length in minutes
37     views: int        # popularity, in millions (used only for sorting/display)
38     channel: str
39     description: str

```

read_integer | original source lines 121-129

```

121 def read_integer(prompt: str, min_val: int, max_val: int) -> int:
122     """Keep asking until the user types a whole number inside [min_val, max_val]."""
123     while True:
124         raw = input(prompt).strip()          # get text from the user
125         if raw.isdigit():                    # check it's only digits (no letters/symbols)
126             value = int(raw)                 # convert text "5" into number 5
127             if min_val <= value <= max_val:  # check it's within the allowed range
128                 return value                 # good input -> stop asking
129         print(f"Invalid input. Enter a number from {min_val} to {max_val}.")

```

recommend_by_category | original source lines 188-201

```

188 def recommend_by_category(catalog: List[Video], watch_later: List[Video]) -> None:
189     """Menu option 1: show videos that match a category typed by the user."""
190     print_header("RECOMMEND BY CATEGORY")
191     print("Available categories: Tech, Music, Fitness, Education, Comedy, Gaming, Vlog")
192     category = input("Enter category: ").strip().lower() # clean up user's text
193
194     # keep only the videos whose category matches what the user typed
195     matches = [v for v in catalog if v.category.lower() == category]
196     if not matches:
197         print("No videos found in that category.")
198         return
199
200     print_video_list(matches)                # show the matching videos
201     add_to_watch_later(matches, watch_later) # offer to save one

```

Full source: [Open youtube_recommendation_assistant.py on GitHub](#).

Code: main program logic

Figure 2. The main loop loads the catalogue, stores the current Watch Later list, accepts menu choices and dispatches the selected function.

main() | original source lines 300-315

```

300 def main() -> None:
301     """Program entry point: load data, then loop showing the menu."""
302     catalog = load_catalog()          # all videos the program knows about
303     watch_later: List[Video] = []    # starts empty, grows as the user saves videos
304     running = True                   # controls the loop below
305
306     print_header("YOUTUBE VIDEO RECOMMENDATION ASSISTANT")
307     print("This mini application demonstrates a simplified YouTube-style")
308     print("recommendation system based on user preferences.")
309     print(f"\nVideos loaded: {len(catalog)}")
310     print("Project topic: YouTube as a disruptive innovation in media & broadcasting.")
311
312     # main loop: show the menu, run the chosen feature, repeat until Exit
313     while running:
314         display_menu()
315         choice = read_integer("Choose an option (1-7): ", MENU_MIN_OPTION, MENU_MAX_OPTION)

```

main() continued | original source lines 316-331

```

316
317     if choice == 1:
318         recommend_by_category(catalog, watch_later)
319     elif choice == 2:
320         recommend_by_mood(catalog, watch_later)
321     elif choice == 3:
322         filter_by_time(catalog, watch_later)
323     elif choice == 4:
324         view_watch_later(watch_later)
325     elif choice == 5:
326         clear_watch_later(watch_later)
327     elif choice == 6:
328         about_youtube_innovation()
329     elif choice == 7:
330         running = False # this stops the while loop, ending the program
331         print("\nThank you for using the YouTube Recommendation Assistant.")

```

Source file

The screenshots show excerpts from **youtube_recommendation_assistant.py**, the source file included in this submission. The complete file is linked on page 4.

Programming language

Python was used for this project with the lecturer's approval.

Program output

Figure 3. Browser screenshot of saved stdout from actual Python runs. It shows category results, a saved video, the 18-minute total, invalid-input handling and exit.

Run 1 inputs: 1, Tech, 1, 1, 4, 7 | category result

RECOMMEND BY CATEGORY

=====

Available categories: Tech, Music, Fitness, Education, Comedy, Gaming, Vlog

Enter category: # Title Category Mood Min Views(M)Channel

1 Building a Startup in 2026 Tech Informative 18 4 TechInsights

A practical breakdown of launching a small startup with limited funds.

2 How AI Is Changing Everyday AppsTech Informative 20 11 TechInsights

An overview of practical AI features appearing in daily-use apps.

3 Coding a Simple Game in a WeekendTech Exciting 28 6 DevWeekend

A hands-on look at building a small game from scratch quickly.

4 Watch this BEFORE you buy a Camera!Tech Informative 12 14 GearReviewer

Key things to check before purchasing a camera, aimed at beginners.

Add one video to Watch Later? 1 = Yes, 2 = No: Enter the video number to save: "Building a Startup in 2026" added to Watch Later.

Run 1 continued | saved video and total duration

MY WATCH LATER LIST

=====

Title Category Mood Min Views(M)Channel

1 Building a Startup in 2026 Tech Informative 18 4 TechInsights

A practical breakdown of launching a small startup with limited funds.

Total: 1 video(s), 18 minutes of watch time.

Run 2 inputs: 0, 7 | invalid menu choice and exit

Choose an option (1-7): Invalid input. Enter a number from 1 to 7.

Choose an option (1-7):

Thank you for using the YouTube Recommendation Assistant.

Execution: Python 3.12.14. Capture date: 2026-09-28. The screenshot presents selected output sections. Full input sequences and unedited stdout files accompany this PDF.

Git history and source links

Figure 4. Actual output of `git log --oneline --graph` from the local repository, followed by the last commit details. The correct option is `--oneline`.

Actual Git command output

```
git log --oneline --graph

* 2d59bf7 Document browser upload and portable Git history
* 5fb7e7d Add CLI verification and document GitHub submission evidence
* 3e0a972 Document usage, inputs, outputs and available Git history
* f5e62e8 Import supplied Python recommendation assistant as baseline

git log -1 --format=fuller

commit 2d59bf7a96f23849bae1da9717673574a8707346
Author: sameer-8o <sammer765bassam@gmail.com>
AuthorDate: Sun Sep 27 21:18:25 2026 +0800
Commit: sameer-8o <sammer765bassam@gmail.com>
CommitDate: Sun Sep 27 21:18:25 2026 +0800

    Document browser upload and portable Git history
```

Submission links

[Source code \(Python file\)](#)

https://github.com/sameer-8o/ldcw6123-youtube-python/blob/main/youtube_recommendation_assistant.py

[Git log \(text evidence\)](#)

<https://github.com/sameer-8o/ldcw6123-youtube-python/blob/main/git-log.txt>

[Complete repository and preserved history](#)

<https://github.com/sameer-8o/ldcw6123-youtube-python>

Scope and access

The recorded Git history starts with the source import, followed by documentation, verification and packaging. The ZIP and Git bundle preserve these local commits; GitHub uploads have a separate history. The source code and Git log are available in the public repository.

This is an evidence section to insert into the final project PDF. The screenshots show browser views of source excerpts and saved execution output.